

C# and ASP.NET – Simplified Practical

Practical 1 – Introduction to C# Programming – Part 1

Q1. Sum of digits

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter the number");
        int n = int.Parse(Console.ReadLine());

        int sum = 0;
        while (n > 0)
        {
            sum = sum + n % 10;
            n = n / 10;
        }
        Console.WriteLine("Sum" + sum);
    }
}
```

Q2. Number pattern

```
using System;

class Program
{
    static void Main()
    {
        for (int i=1; i<=5; i++)
        {
            for(int j=1; j<=i; j++)
            {
                Console.Write(j + "");
            }
            Console.WriteLine();
        }
    }
}
```

3. Pattern — Star Triangle

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)           // rows
        {
```

```

        for (int j = 1; j <= i; j++) // print stars
            Console.Write("* ");

        Console.WriteLine();
    }
}

```

Remember: Change `Console.Write("* ")` to another symbol if asked.

4. Pattern — Reverse Star Triangle

```

using System;

class Program
{
    static void Main()
    {
        for (int i = 5; i >= 1; i--) // 5 to 1
        {
            for (int j = 1; j <= i; j++)
                Console.Write("* ");

            Console.WriteLine();
        }
    }
}

```

Remember: Only the outer loop changes direction: `i--`.

5. Pattern — Number Pyramid

```

using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            for (int s = 1; s <= 5 - i; s++)
                Console.Write(" "); // spaces

            for (int j = 1; j <= i; j++)
                Console.Write(j + " "); // numbers

            Console.WriteLine();
        }
    }
}

```

Practical 2 – Introduction to C# Programming – Part 2

Q1. Diagonal sum, border sum and maximum of a 3×3 matrix

```
using System;

class Program
{
    static void Main()
    {
        int[,] a = new int[3, 3];
        Console.WriteLine("Enter 9 numbers :");

        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                a[i, j] = int.Parse(Console.ReadLine());
            }
        }

        int diagonal = 0;
        int border = 0;
        int max = a[0, 0];

        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                if (i == j)
                    diagonal += a[i, j];
                if (i == 0 || i == 2 || j == 0 || j == 2)
                    border += a[i, j];
                if (a[i, j] > max)
                    max = a[i, j];
            }
        }

        Console.WriteLine("Diagonal Sum =" + diagonal);
        Console.WriteLine("Border Sum =" + border);
        Console.WriteLine("Maximum =" + max);
    }
}
```

Practical 3 – Basic LINQ Implementation in C#

Use the productdata.xml file from the original journal, or the simplified sample below.

Product.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Products>
    <Product>
        <pid>101</pid>
        <pname>Samsung</pname>
        <price>50000</price>
        <dom>10-01-2026</dom>
    </Product>
    <Product>
```

```

        <pid>102</pid>
        <pname>Iphone</pname>
        <price>100000</price>
        <dom>10-10-2026</dom>
    </Product>

    <Product>
        <pid>103</pid>
        <pname>DELL</pname>
        <price>55000</price>
        <dom>05-01-2026</dom>
    </Product>

    <Product>
        <pid>104</pid>
        <pname>HP</pname>
        <price>60000</price>
        <dom>10-08-2026</dom>
    </Product>
</Products>

```

Program.cs

```

using System;
using System.Linq;
using System.Xml.Linq;

class Program
{
    static void Main()
    {
        XmlDocument doc =
XDocument.Load("C:\\Users\\User\\source\\repos\\Practical_Exam\\Practical_Exam\\p
roduct.xml");

        var products = doc.Descendants("Product");

        // ----- PRICE QUERIES -----

        Console.WriteLine("ALL PRODUCTS");

        foreach (var p in products)
        {
            Console.WriteLine(
                p.Element("pname").Value + " " +
                p.Element("price").Value);
        }

        Console.WriteLine("\nPRICE > 55000");

        var result = products.Where(p =>
            (int)p.Element("price") > 55000);

        foreach (var p in result)
            Console.WriteLine(
                p.Element("pname").Value + " " +
                p.Element("price").Value);

        Console.WriteLine("\nPRICE < 65000");
    }
}

```

```

var result2 = products.Where(p =>
    (int)p.Element("price") < 65000);

foreach (var p in result2)
    Console.WriteLine(
        p.Element("pname").Value + " " +
        p.Element("price").Value);

Console.WriteLine("\nNOT DELL");

var result3 = products.Where(p =>
    (string)p.Element("pname") != "DELL");

foreach (var p in result3)
    Console.WriteLine(p.Element("pname").Value);

Console.WriteLine("\nPRICE > 50000 AND NOT IPHONE");

var result4 = products.Where(p =>
    (int)p.Element("price") > 50000 &&
    (string)p.Element("pname") != "Iphone");

foreach (var p in result4)
    Console.WriteLine(p.Element("pname").Value);

// ----- DATE QUERIES -----

Console.WriteLine("\nDATE > 10-01-2026");

var r1 = products.Where(p =>
    DateTime.ParseExact(
        (string)p.Element("dom"),
        "dd-MM-yyyy",
        null)
    >
    DateTime.ParseExact(
        "10-01-2026",
        "dd-MM-yyyy",
        null));

foreach (var p in r1)
    Console.WriteLine(p.Element("pname").Value);

Console.WriteLine("\nDATE > 01-07-2026 AND PRICE > 40000");

var r2 = products.Where(p =>
    (int)p.Element("price") > 40000 &&
    DateTime.ParseExact(
        (string)p.Element("dom"),
        "dd-MM-yyyy",
        null)
    >
    DateTime.ParseExact(
        "01-07-2026",
        "dd-MM-yyyy",
        null));

```

```

        foreach (var p in r2)
            Console.WriteLine(p.Element("pname").Value);

        Console.WriteLine("\nDATE BETWEEN 01-07-2026 AND 01-10-2026");

        var r3 = products.Where(p =>
            DateTime.ParseExact(
                (string)p.Element("dom"),
                "dd-MM-yyyy",
                null)
            >=
            DateTime.ParseExact(
                "01-07-2026",
                "dd-MM-yyyy",
                null)
            &&
            DateTime.ParseExact(
                (string)p.Element("dom"),
                "dd-MM-yyyy",
                null)
            <=
            DateTime.ParseExact(
                "01-10-2026",
                "dd-MM-yyyy",
                null));

        foreach (var p in r3)
            Console.WriteLine(p.Element("pname").Value);
    }
}

```

Practical 5 – Inheritance and Hierarchy – Part 1

Constructor and constructor chaining

```

using System;

class A
{
    public A()
    {
        Console.WriteLine("A Default Constructor");
    }
    public A(string name ) : this()
    {
        Console.WriteLine("A Parameterized Constructor");
    }
}

class B : A
{
    public B():base ("trial")
    {
        Console.WriteLine("B Constructor");
    }
}

class Program :B
{
    static void Main()
    {
    }
}

```

```

    {
        Program t = new Program();
    }
}

```

Practical 6 – Inheritance and Hierarchy – Part 2

Example 1 – virtual and override

```

using System;

class A { public virtual void Display() => Console.WriteLine("In A Display");}
class B:A { public override void Display() => Console.WriteLine("In B Display");}
class C:B { public new void Display() => Console.WriteLine("In C Display"); }

class Program
{
    static void Main()
    {
        A a = new C();
        a.Display();
    }
}

```

Example 2 – new

```

using System;

class A { public virtual void Display() => Console.WriteLine("In A Display");}
class B:A { public void Display() => Console.WriteLine("In B Display");}

class Program
{
    static void Main()
    {
        A obj = new B();
        obj.Display();
    }
}

```

Practical 7 – Structures in C#

Q1. Store and retrieve student data

```

using System;

struct Student
{
    public int rollno;
    public string name;
    public int age;
    public string course;
    public int marks;
}

class Program
{

```

```

static void Main()
{
    Student[] s = new Student[3];

    for(int i=0; i<3;i++)
    {
        Console.Write("Roll no :");
        s[i].rollno=int.Parse(Console.ReadLine());

        Console.Write("Name :");
        s[i].name = Console.ReadLine();

        Console.Write("Age :");
        s[i].age = int.Parse(Console.ReadLine());

        Console.Write("Course :");
        s[i].course =(Console.ReadLine());

        Console.Write("Marks :");
        s[i].marks = int.Parse(Console.ReadLine());
    }

    Console.WriteLine("\nStudent Details");
    for (int i=0;i<3;i++)
    {
        Console.WriteLine("-----");
        Console.WriteLine("Roll: " + s[i].rollno);
        Console.WriteLine("Name: " + s[i].name);
        Console.WriteLine("Age: " + s[i].age);
        Console.WriteLine("Course: " + s[i].course);
        Console.WriteLine("Marks: " + s[i].marks);
        Console.WriteLine("-----");
    }
}
}

```

Q2. Store and retrieve employee data

```

using System;

struct Employee
{
    public int eno;
    public string name;
    public int age;
    public int salary;
}

class Program
{
    static void Main()
    {
        Employee[] e = new Employee[5];

        for (int i = 0; i < 5; i++)
        {
            Console.Write("Employee No: ");
            e[i].eno = int.Parse(Console.ReadLine());

            Console.Write("Name: ");
            e[i].name = Console.ReadLine();

```

```

        Console.WriteLine("Age: ");
        e[i].age = int.Parse(Console.ReadLine());

        Console.WriteLine("Salary: ");
        e[i].salary = int.Parse(Console.ReadLine());
    }

    Console.WriteLine("\nSalary > 60000");
    for (int i = 0; i < 5; i++)
    {
        if (e[i].salary > 60000)
            Console.WriteLine(e[i].name + " " + e[i].salary);
    }
    Console.WriteLine("\nAge > 30");
    for (int i = 0; i < 5; i++)
    {
        if (e[i].age > 30)
            Console.WriteLine(e[i].name + " " + e[i].age);
    }
    int max = e[0].salary;
    string name = e[0].name;

    for (int i = 1; i < 5; i++)
    {
        if (e[i].salary > max)
        {
            max = e[i].salary;
            name = e[i].name;
        }
    }
    Console.WriteLine("\nHighest Salary:");
    Console.WriteLine(name + " " + max);
}
}

```

Practical 8 – ASP.NET Razor Pages CRUD

- **Create the Project:** Open Visual Studio → **Create a new project** → Select **ASP.NET Core Web App (Razor Pages)** → Set name to **RazorCrudDemo** → Choose .NET version → Click **Create**.
- **Create the Model Folder:** Solution Explorer → Right-click project → **Add** → **New Folder** → Name it **Models**.
- **Create the Employee Class:** Right-click **Models** → **Add** → **Class** → Name it **Employee.cs**.
- **Add Model Code:** Paste the model code (with **Id**, **Name**, **Age**, **Salary** attributes) into **Employee.cs**.
- **Build Solution:** Press **Ctrl + Shift + B** (or **Build** → **Build Solution**) and ensure build succeeds.

- **Create Pages Folder:** Solution Explorer → Right-click **Pages** → **Add** → **New Folder** → Name it **Employees**.
- **Add Scaffolded Item:** Right-click **Employees** folder → **Add** → **New Scaffolded Item...**
- **Select Scaffolder:** Select **Razor Pages using Entity Framework (CRUD)** → Click **Add**.
- **Set Model Class:** Set **Model class** to **Employee** (**RazorCrudDemo.Models**).
- **Set Data Context:** Click the + button next to **Data context class** → Accept default name (**RazorCrudDemoContext**) → Select database provider (e.g., SQL Server) → Click **Add**.

Quick Revision – What to Memorize

- Practical 1: while loop, % 10, / 10, nested for loops
- Practical 2: 2D array, i == j, border condition, maximum
- Practical 3: XDocument, Descendants(), Where(), foreach
- Practical 4: DateTime.ParseExact() and date conditions
- Practical 5: this() calls same-class constructor; base() calls parent constructor
- Practical 6: virtual, override and new
- Practical 7: struct, array of structures, if conditions
- Practical 8: Model → Repository → Razor Pages → CRUD

1. Master Pages in ASP.NET Web Forms

Master Pages allow you to define a consistent layout (header, sidebar, footer) across multiple pages.

Step 1: Create the Master Page (**Site.Master**)

1. In Visual Studio, right-click project → **Add** → **New Item** → **Master Page**.
2. Define common layout elements along with `<asp:ContentPlaceHolder>` regions.

HTML

```
<%@ Master Language="C#" AutoEventWireup="true"
CodeBehind="Site.master.cs" Inherits="WebApp.Site" %>
```

```

<!DOCTYPE html>
<html>
<head runat="server">
    <title>My Website</title>
    <asp:ContentPlaceholder ID="head"
runat="server"></asp:ContentPlaceholder>
</head>
<body>
    <form id="form1" runat="server">
        <header>
            <h1>My College Portal</h1>
            <hr />
        </header>

        <!-- Main Dynamic Content Area -->
        <div>
            <asp:ContentPlaceholder ID="MainContent" runat="server">
                </asp:ContentPlaceholder>
            </div>

            <footer>
                <hr />
                <p>&copy; 2026 Practical Exam</p>
            </footer>
        </form>
    </body>
</html>

```

Step 2: Create a Content Page (Home .aspx)

1. Right-click project → **Add** → **New Item** → **Web Form with Master Page**.
2. Select Site.Master.

HTML

```

<%@ Page Title="Home Page" Language="C#" MasterPageFile="~/Site.Master"
AutoEventWireup="true" CodeBehind="Home.aspx.cs" Inherits="WebApp.Home"
%>

<asp:Content ID="Content1" ContentPlaceHolderID="head" runat="server">
</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="MainContent"
runat="server">
    <h2>Welcome to Home Page</h2>
    <p>This content is injected into the Master Page layout.</p>
</asp:Content>

```

2. MVC (Model-View-Controller) in ASP.NET

MVC splits application logic into three interconnected parts: **Model** (Data), **View** (UI), and **Controller** (Business Logic).

Step 1: Model (Models/Student.cs)

Right-click Models folder → **Add** → **Class**.

C#

```
namespace MvcDemo.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
    }
}
```

Step 2: Controller (Controllers/StudentController.cs)

Right-click Controllers folder → **Add** → **Controller** → **MVC Controller - Empty**.

C#

```
using Microsoft.AspNetCore.Mvc;
using MvcDemo.Models;
using System.Collections.Generic;

namespace MvcDemo.Controllers
{
    public class StudentController : Controller
    {
        // GET: /Student/Index
        public IActionResult Index()
        {
            List<Student> students = new List<Student>
            {
                new Student { Id = 101, Name = "Aarav", Course = "BSc
IT" },
                new Student { Id = 102, Name = "Riya", Course = "BCA" }
            };

            return View(students); // Passes model data to the View
        }
    }
}
```

Step 3: View (Views/Student/Index.cshtml)

Right-click inside the Index() method in the controller → **Add View** → **Razor View** (or manually create Index.cshtml under Views/Student/).

HTML

```
@model IEnumerable<MvcDemo.Models.Student>
```

```

<h2>Student List</h2>

<table border="1" cellpadding="5">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Course</th>
    </tr>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Name</td>
            <td>@item.Course</td>
        </tr>
    }
</table>

```

Quick Revision Notes for Practical Viva

- **Master Page (.master):** Uses <asp:ContentPlaceHolder> to mark regions where child pages (.aspx) inject content using <asp:Content ContentPlaceHolderID="...">.
- **MVC Pattern:**
 - **Model:** Holds data structures and business domain logic.
 - **View:** Handles UI rendering using Razor syntax (@).
 - **Controller:** Handles request processing, updates Model, and renders appropriate View.